industrial it-solutions

Gold Microsoft Partner Application Development

IODA Architecture Model

DECOUPLING CONCERNS EVEN MORE BY GETTING RID OF FUNCTIONAL DEPENDENCIES

Agenda

- Identifying **Problems/Critics** of Common Architecture Models
  - Monolith code example
  - Trying to solve with layers
  - Decoupling of layers
  - Layers in shells
- IODA Architecture Model

Code Example - Requirements

Write a program that

- Counts all words in a text
- Counts all distinct words
- Consider a stop word list (separate file to exclude words)
- Read text from user input or file
  - Use the file name as a program argument

Just a simple example that has some user interface (console), business logic and data access

Do you already have a basic code, architecture, or system design in mind?

In the beginning there was the Monolith without Structure

```
internal class Program
{
    public static void Main(string[] args) {
        var stopwords = new string[0];
        if (File.Exists("stopwords.txt"))
            stopwords = File.ReadAllLines("stopwords.txt");

        var text = "";
        if (args.Length > 0)
            text = File.ReadAllText(args[0]);
        else {
            Console.Write("Text eingeben: ");
            text = Console.ReadLine();
            if (string.IsNullOrWhiteSpace(text)) return;
        }

        var candidateWords = text.Split(new[] {' ', '\t', '\n', '\r'},
                                        StringSplitOptions.RemoveEmptyEntries);
        var countTotal = 0;
        var countDistinct = 0;
        var stopwordsDirectory = new HashSet<string>(stopwords);
        var distinctWords = new HashSet<string>();
        foreach (var wortkandidat in candidateWords) {
            foreach(var m in Regex.Matches(wortkandidat, @"[\w\-]*"))
                if (!string.IsNullOrWhiteSpace(m.ToString())) {
                    var word = m.ToString();

                    if (!stopwordsDirectory.Contains(word)) {
                        countTotal++;

                        if (!distinctWords.Contains(word)) {
                            distinctWords.Add(word);
                            countDistinct++;
                        }
                    }
                }
        }

        Console.WriteLine($"{countTotal} Worte, davon {countDistinct} verschieden");
    }
}
```

- It is functional correct
- Understandable? Why Not?
- Testable?

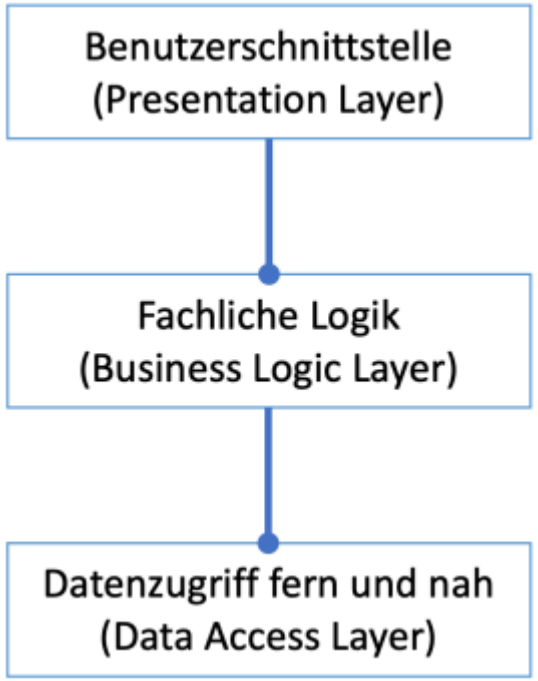
Can you identify all the responsibilities?

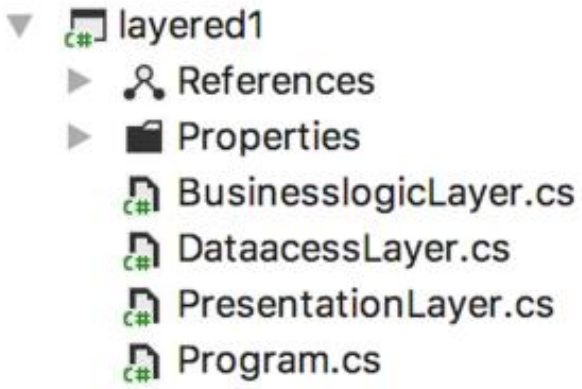
Note: Regex checks if it is an alphanumeric word including dash.

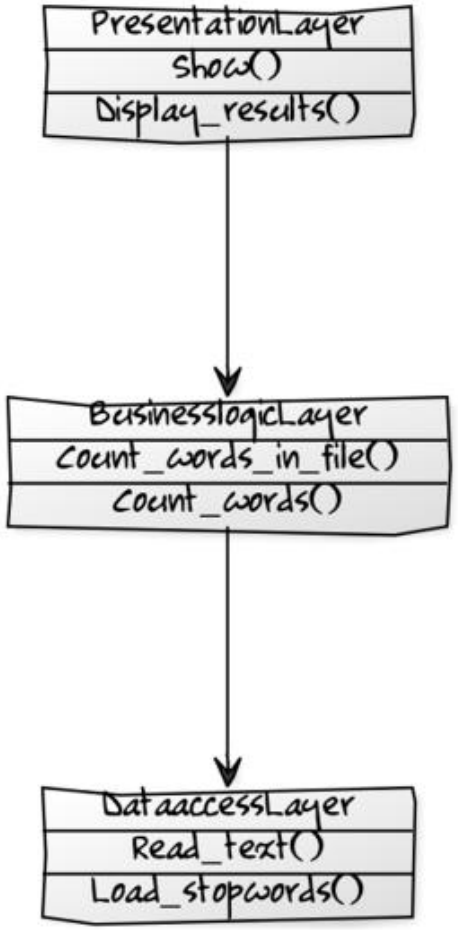
Mixed / Confused Responsibilities

```
internal class Program
{
    public static void Main(string[] args) {
        var stopwords = new string[0];
        if (File.Exists("stopwords.txt"))
            stopwords = File.ReadAllLines("stopwords.txt");

        var text = "";
        if (args.Length > 0)
            text = File.ReadAllText(args[0]);
        else {
            Console.Write("Text eingeben: ");
            text = Console.ReadLine();
            if (string.IsNullOrWhiteSpace(text)) return;
        }

        var candidateWords = text.Split(new[] {' ', '\t', '\n', '\r'},
                                        StringSplitOptions.RemoveEmptyEntries);
        var countTotal = 0;
        var countDistinct = 0;
        var stopwordsDirectory = new HashSet<string>(stopwords);
        var distinctWords = new HashSet<string>();
        foreach (var wortkandidat in candidateWords) {
            foreach(var m in Regex.Matches(wortkandidat, @"[\w\-]*"))
                if (!string.IsNullOrWhiteSpace(m.ToString())) {
                    var word = m.ToString();

                    if (!stopwordsDirectory.Contains(word)) {
                        countTotal++;

                        if (!distinctWords.Contains(word)) {
                            distinctWords.Add(word);
                            countDistinct++;
                        }
                    }
                }
        }

        Console.WriteLine($"{countTotal} Worte, davon {countDistinct} verschieden");
    }
}
```

Splitting up the Monolith in Layers – Layered Architecture







Improved:

- Understandability
- Clarity of responsibilities and relations

Splitting up the Monolith in Layers – Layered Architecture

```
class BusinesslogicLayer
{
    public static (int countTotal, int countDistinct) Count_words_in_file(string filename) {
        var text = DataacessLayer.Read_text(filename);
        return Count_words(text);
    }

    public static (int countTotal, int countDistinct) Count_words(string text) {
        var stopwordsDirectory = DataacessLayer.Load_stopwords();

        var candidateWords = text.Split(new[] {' ', '\t', '\n', '\r'},
                                        StringSplitOptions.RemoveEmptyEntries);
        var countTotal = 0;
        var countDistinct = 0;

        var distinctWords = new HashSet<string>();
        foreach (var wordCandidate in candidateWords) {
            foreach(var m in Regex.Matches(wordCandidate, @"[\w\-]*"))
                if (!string.IsNullOrWhiteSpace(m.ToString())) {
                    var word = m.ToString();

                    if (!stopwordsDirectory.Contains(word)) {
                        countTotal++;

                        if (!distinctWords.Contains(word)) {
                            distinctWords.Add(word);
                            countDistinct++;
                        }
                    }
                }
        }

        return (countTotal, countDistinct);
    }
}
```

The **rough** responsibilities are basically nicely separated

Dependencies neatly aligned

But actual business logic is still

not easy to test

Functional Dependency exists**:**

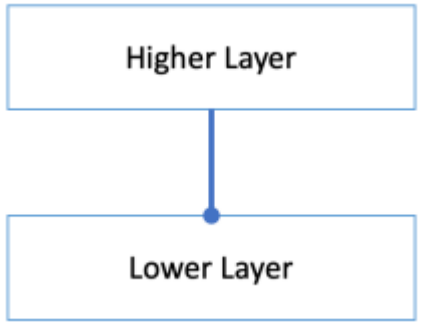
BL calls other logic from DAL

Decoupling of Layers

- DIP – Dependency Inversion Principle
  - High-level modules should not depend on low-level modules. **Both** should depend on abstractions.
  - Abstractions should not depend on details. Details should depend on abstractions.
- Enables focused testing of a layer

Trick: Split **compile/design** time dependency from **runtime** dependency

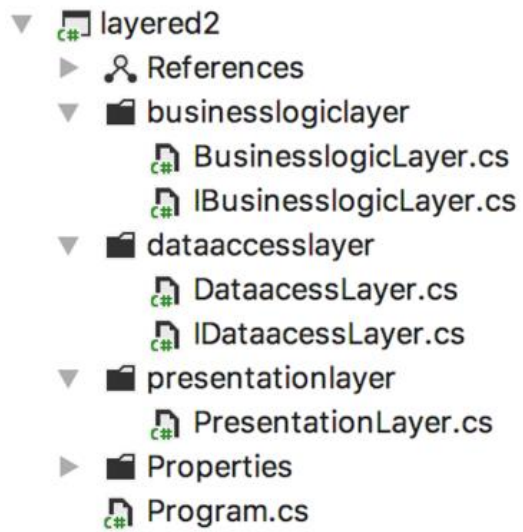
without DIP



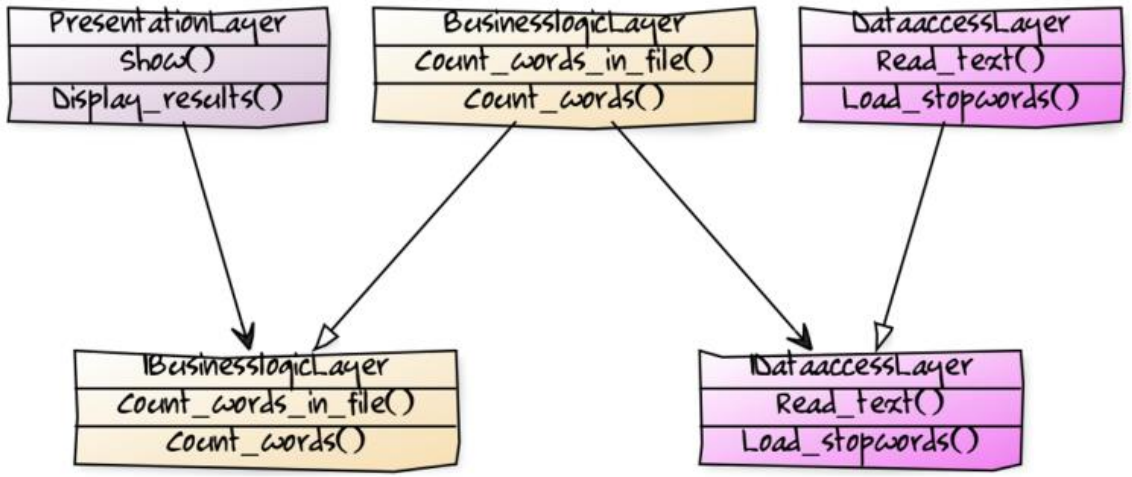
with DIP

Decoupling of Layers

Layered Architecture with DIP

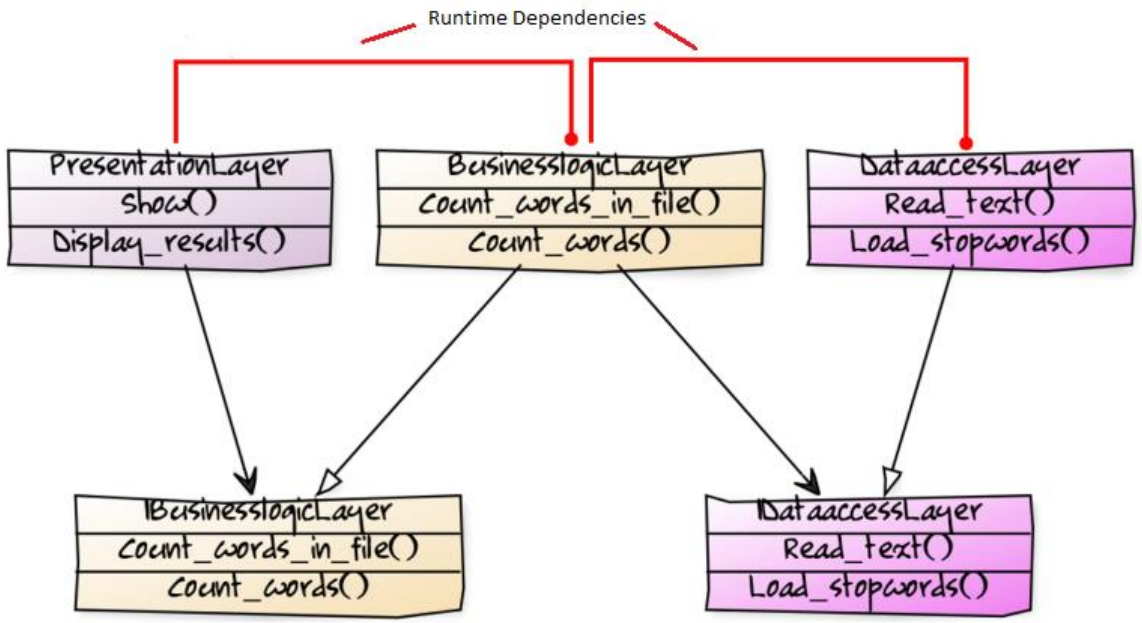


Interfaces are decoupling layers



Decoupling of Layers

- Inversion of Control – IoC
- Inject concrete dependency during runtime




```
internal class Program
{
    public static void Main(string[] args) {
        var dal = new DataaccessLayer();
        var bl = new BusinesslogicLayer(dal);
        var pl = new PresentationLayer(bl);

        pl.Show(args);
    }
}
```

```
class BusinesslogicLayer : IBusinesslogicLayer
{
    private readonly IDataaccessLayer _dal;

    public BusinesslogicLayer(IDataaccessLayer dal) { _dal = dal; }


    public (int countTotal, int countDistinct) Count_words_in_file(string filename) {
        var text = _dal.Read_text(filename);
        return Count_words(text);
    }

    public (int countTotal, int countDistinct) Count_words(string text) {
        var stopwordsDirectory = _dal.Load_stopwords();
```

- Diagram not so easy anymore
- With DIP dependencies are different during compile time and runtime
- Additional artifacts -> Interfaces
- Functional Dependency still exists

=> **Complexity increased for testability**

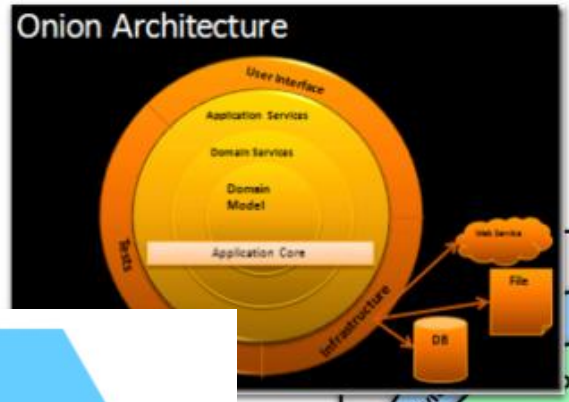
Why…?

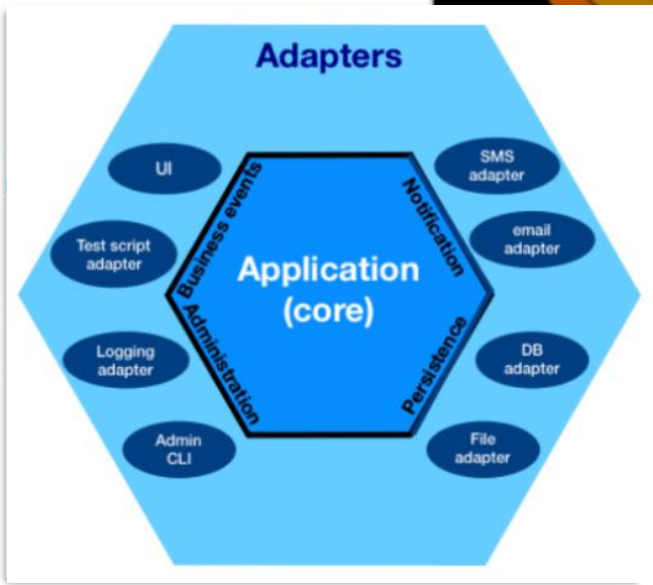
- … should the BL depend on the DAL?
- … should the BL know how to load and save data?
- … should the BL be responsible to control the data access?

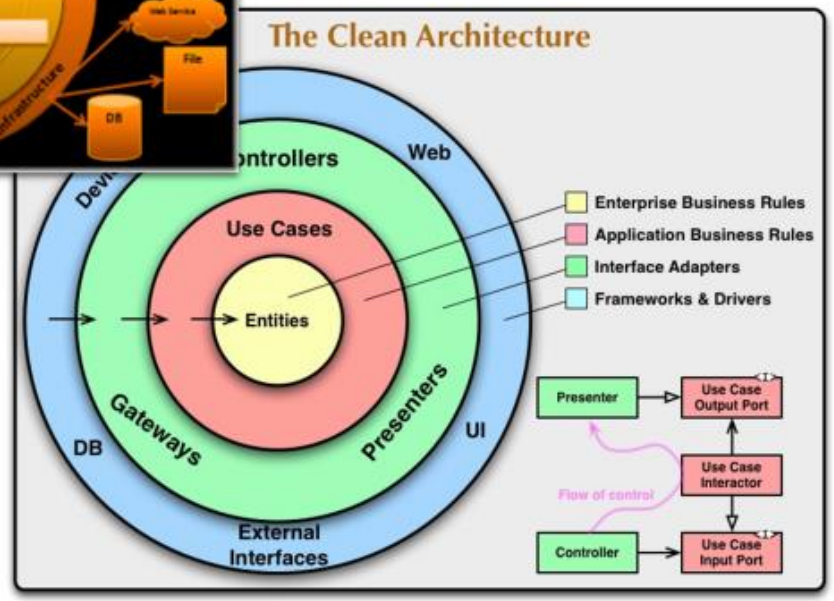
=> There seems to be some **confusion of responsibilities**!?

Layers in Shells – To the Rescue?

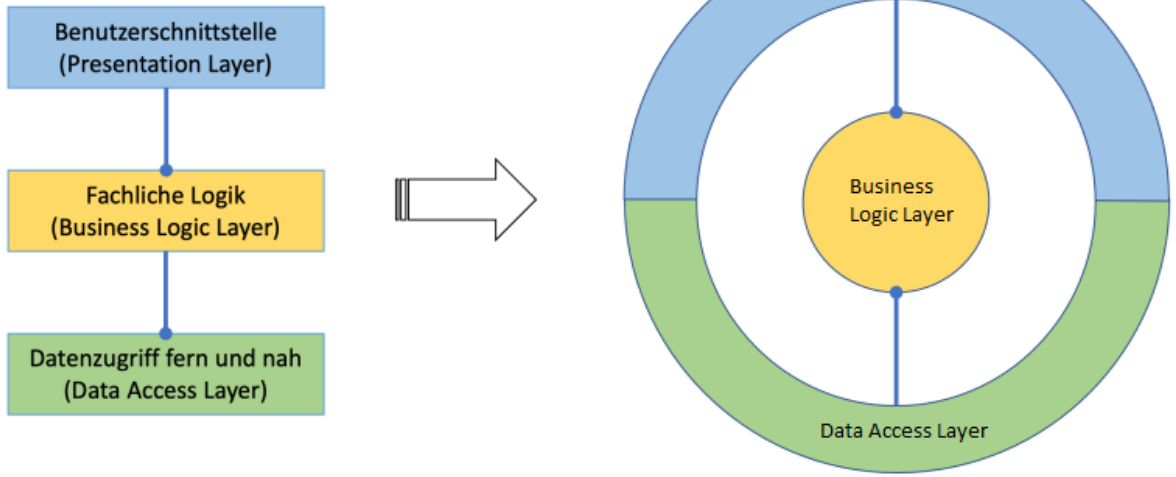
- In 2005 the industry realized there is something wrong
- Concentric architecture patterns came up
  - Hexagonal Architecture (HA)
  - Onion Architecture (OA)
  - Clean Architecture (CA)
    from Robert C. Martin Clean Code Author
- Try a different organization of responsibilities; different kinds
- Core differs from outer parts of the software
- Still clear direction of dependencies like in Layered Architecture





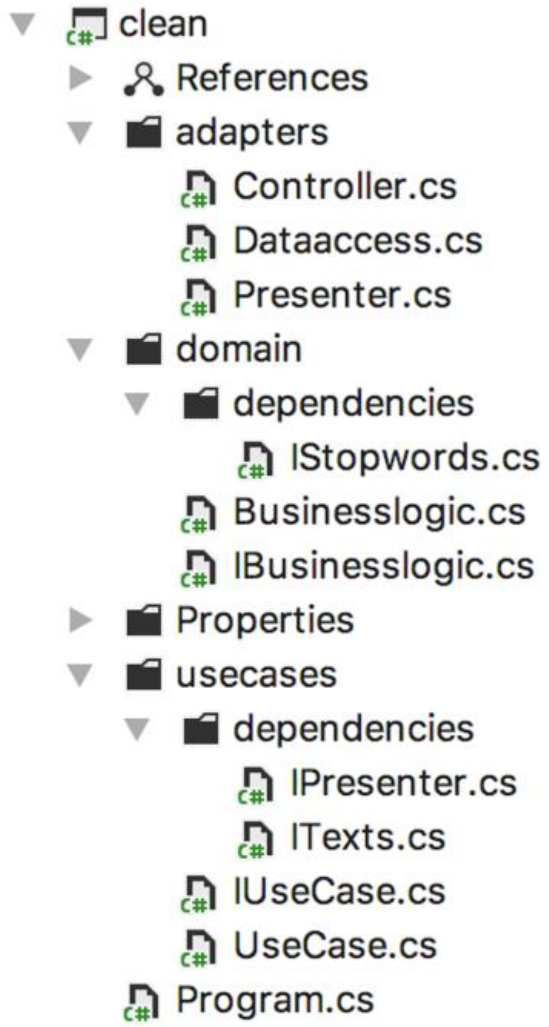


Layers in Shells – Problems solved?



- Dependencies direction from outside -> inside
- BL does not control the DAL, it **seems** so
- The further out a shell, the more
  - it deals with technical/peripheral details
  - volatile it is
- The further inside a responsibility, the more
  - central it is
  - timeless, constant and quiet it is
  - increases the policies/rules -> **sounds good**
  - increases the abstraction -> **good idea?**

Layers in Shells - Clean Architecture (CA)



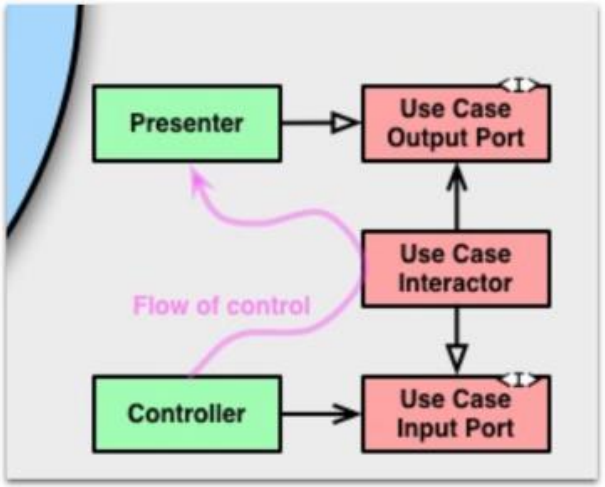
- Adapters
  - Outer shell
  - Hardware/Resources access (keyboard, display, hard disk, network, …)
  - Presentation Layer Logic is now split
    - Controller: delegates user actions to Use Cases shell
    - Presenter: used by Use Cases shell to display results
- Use Cases
  - Shell between core/domain and outer shell
  - Orchestrate the flow of data to and from the domain
  - Defines it`s (compile time) dependencies as abstractions to access data (ITexts) and UI (IPresenter)
- Domain
  - Core
  - Entities, Services
  - Defines it`s (compile time) dependency as abstraction to access data (IStopwords)

Layers in Shells - Clean Architecture (CA) – Clean?

```
class UseCase : IUseCase
{
    private readonly IBusinesslogic _bl;
    private readonly ITexts _txt;
    private readonly IPresenter _ui;

    public UseCase(IBusinesslogic bl, ITexts txt, IPresenter ui) {
        _bl = bl;
        _txt = txt;
        _ui = ui;
    }


    public void Count_words_in_file(string filename) {
        var text = _txt.Retrieve(filename);
        Count_words(text);
    }

    public void Count_words(string text) {
        var (countTotal, countDistinct) = _bl.Count_words(text);
        _ui.Display_results(countTotal, countDistinct);
    }
}
```

Orchestration in Use Case

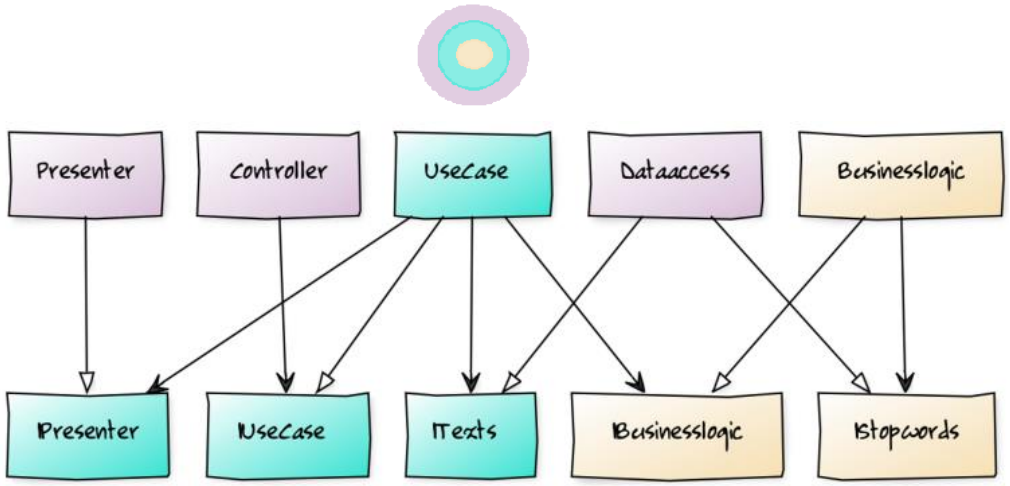
- Retrieve data from Controller, call BL, display results



- **CA requirement** satisfied -> **compile** time **dependencies** point **from outside to inside!**
- At runtime…? (see later)
- **Maybe Complex**?

Layers in Shells - Clean Architecture (CA) – Clean?

```
class Businesslogic : IBusinesslogic
{
    private readonly IStopwords _dal;

    public Businesslogic(IStopwords dal) { _dal = dal; }


    public (int countTotal, int countDistinct) Count_words(string text) {
        var stopwordsDirectory = _dal.Retrieve();

        var candidateWords = text.Split(new[] {' ', '\t', '\n', '\r'},
                                        StringSplitOptions.RemoveEmptyEntries);
        var countTotal = 0;
        var countDistinct = 0;
```



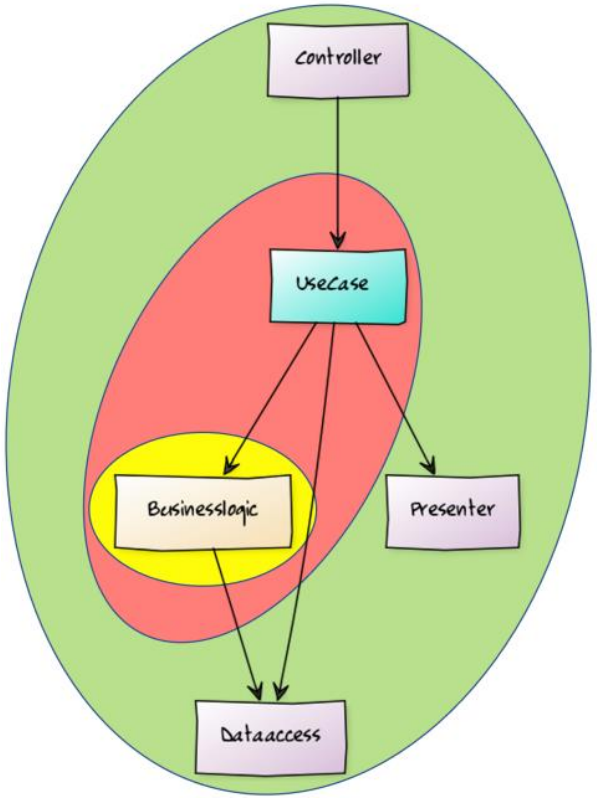
Dependencies clean and decoupled with DIP

- **Functional dependency** still exists
  - logic from DAL located in logic from BL
- **CA requirement** satisfied -> **compile** time **dependencies** point **from outside to inside!**
- But at **run time**: call from an **inner** shell to **outer** shell
- Compared to Layered Architecture:
  - 5 classes instead of 3
  - 5 interfaces instead of 3, must be defined on two levels
    - Core
    - Use Case shell

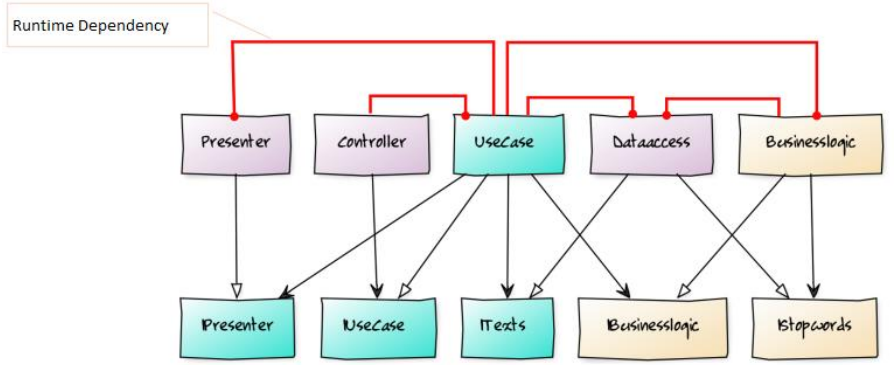
„Much of the complexity in that diagram was intended to make sure that the dependencies between the components pointed in the correct direction." Robert C. Martin

=> The price for **clean** architecture is **complexity**? Really?

Layers in Shells - Clean Architecture (CA) – Clean?



- At design time the dependencies looked cleanly aligned
- But the **truth** at **runtime** is **inner** shells are **depending on outer** shells



Conclusion:

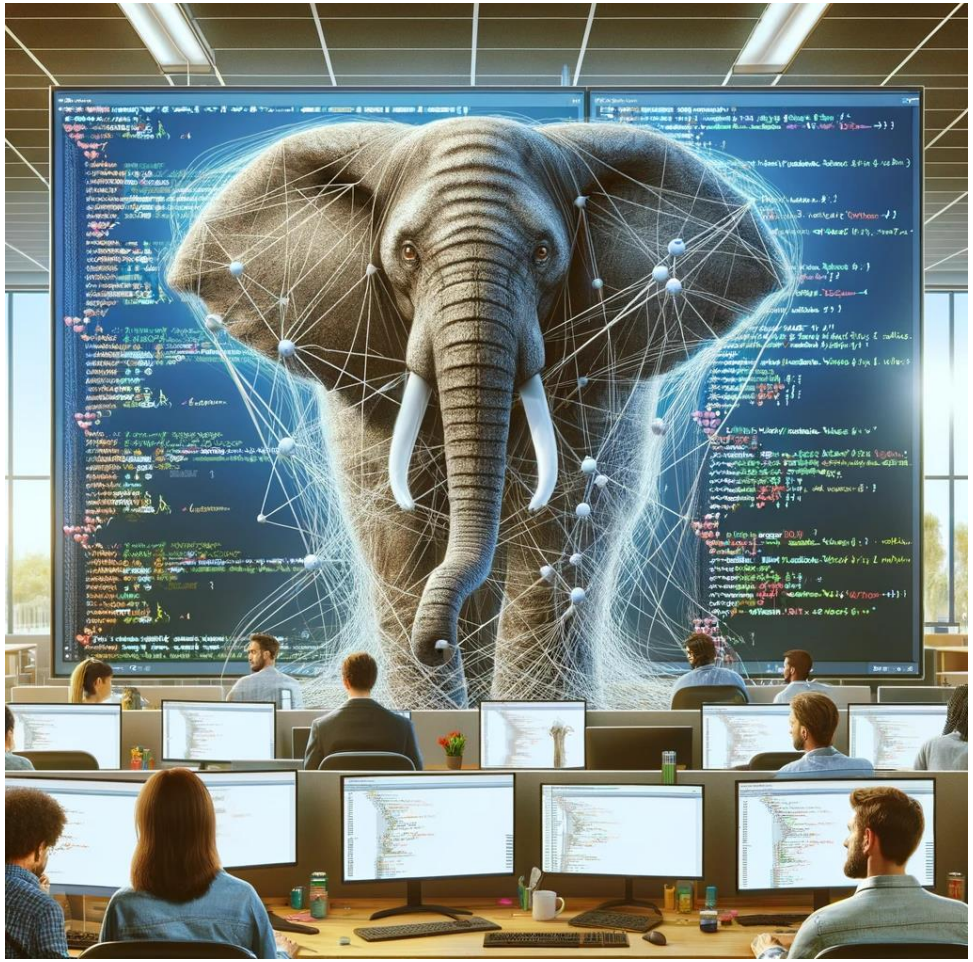
- **Good**:
  - stable rules and policies in the inner core
  - volatility / communication with outside world is located at the outer shells
- **Bad**:
  - too much **focus** on **compile time -> complexity**
  - more artefacts (interfaces, classes, files)
  - testing more complex (setup mocks, injection)

The IODA Architecture Model

Goals for an improved Architecture Pattern

- Keep the advantages from previous patterns
  - Have defined **default responsibilities,** and to be able to **separate** them
  - Improve **understandability** and **testability**
- **Awareness** of cleanly directed dependencies
- Fix the disadvantages from previous patterns
  - **Avoid functional dependencies** that are hard to test
  - **Avoid** additional **complexity** because of decoupling efforts

Functional Dependencies - The Elephant in the Room



If in a function,

LOGIC and **calls** to **other functions** of **your** sources are **co-located / mixed** in the **same function**, then that's a

FUNCTIONAL DEPENDENCY.

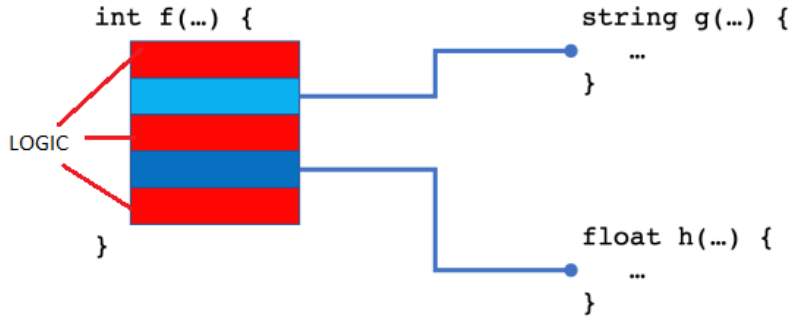
When a **Mock** is required to test **LOGIC** of your codebase.

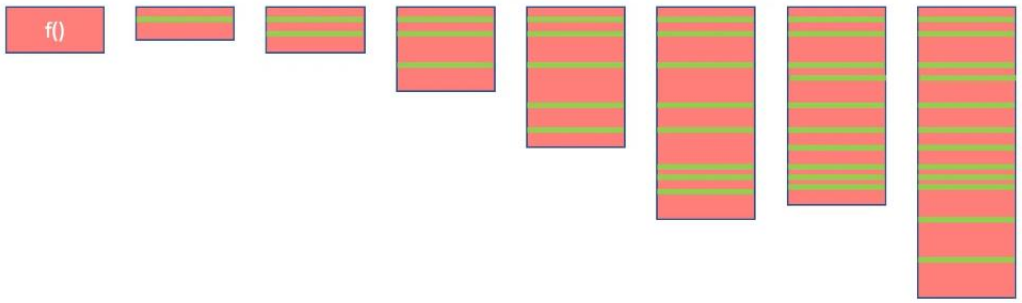
What is **LOGIC?**

- Code where the **action happens**
- Creates software **behavior** for the user (Customer don´t care about code structure, comments, whitespace or variable declarations or function definitions or classes)
  - Transformations / Operators: <, ++, -
  - Control structures: if-else, for, try-catch
  - I/O or general API-Calls: Console.Write(), File.OpenRead()
- Another definition: Functions in binary form (black box)
  - Operators of your programming language
  - 3rd party packages
  - Package references of your software project (not DLLs/csproj, since that source code is near and could easily create FDs, but Packages need some effort to test and build, changes much less, is more stable and versioned)

Functional Dependencies – Todays State of the Art?

Switching between logic and function calls





A function slowly growing by adding functional dependencies.

- Each LOGIC **easy testable?**
  - DIP and mocks to the rescue?
  - Effort for new Interface
  - Effort to code + configure the mock
  - Inject mock (IoC)
  - Increased **complexity**
- When test **fails**, in **which** part of the **LOGIC** blocks?

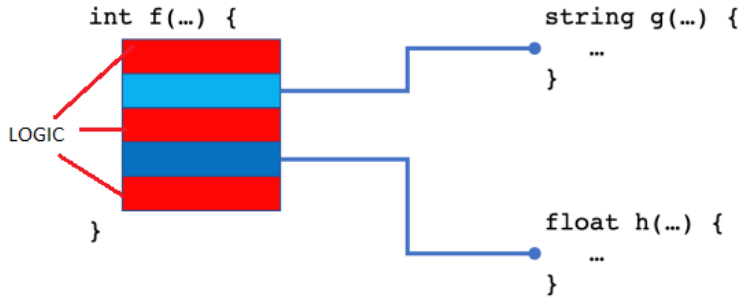
Or maybe an error in the mock?

FDs contradict clean SW-Development

- Hurts **SRP**
  - Create behavior via **Operations** in **LOGIC**
  - **Integration** of other functions
- Hurts **SLA**
  - By nature, logic is on a lower abstraction level than function calls
  - => Bad understandability

=> The classic architecture models do not address this problem

Functional Dependencies – Formal Responsibilities / Testability

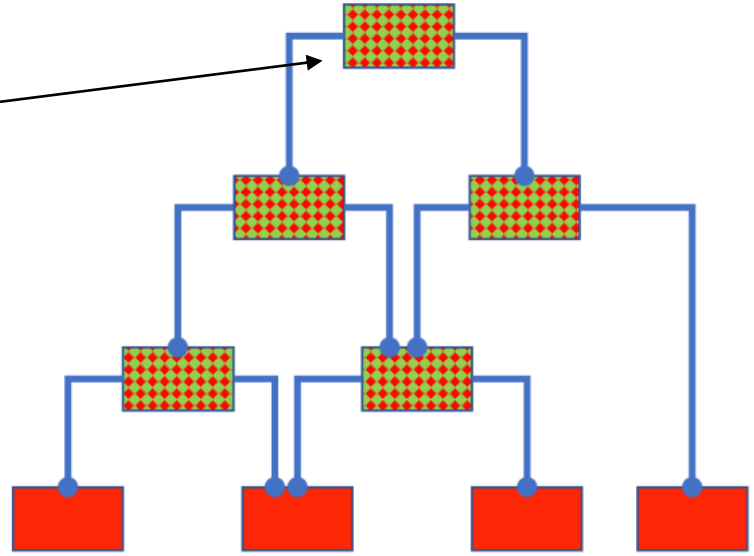


- Extract method does not help
  - Just deepens the logic
  - Harder debugging sessions
  - Harder to see how behavior is produced
- At least n + 2 responsibilities (formal code structure, not regarding content)
  - **n**: the number of FDs
    -> Logic before each call (g, h)
  - **+1**: Logic after last call
  - **+1**: Integration of function calls

- **Testability**
  - Depends on number of FDs
  - The more responsibilities, the more difficult to test
  - Testability = 1 / Number of Responsibilities
    - 1 is best
    - Near 0 is difficult

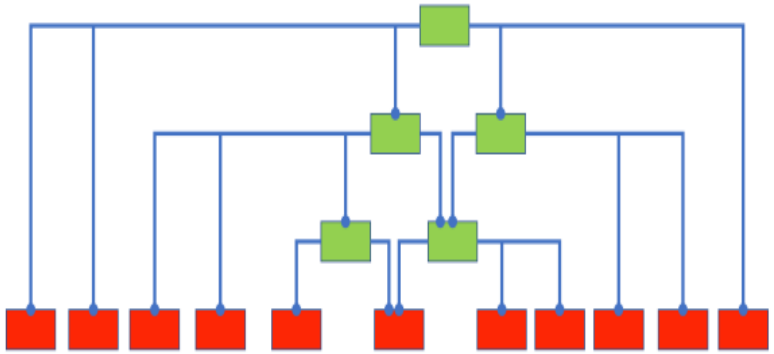
Functional Dependencies

- Common/classic architecture models
  - **Functions** are not a concern. They are **all** from the **same kind**.
  - Logic is not a concern
  - Have **hybrid functions** with two responsibilities (mix of operation (logic) and integration), only a few operations at bottom
    - Always hard to test. DIP/IoC do not help.



New Architecture Pattern - Resolve Functional Dependencies

- Of course, functions are needed and are necessary to resolve FDs
- **Operations**
  - Functions **just** containing **logic**
  - Do **not call** other **functions** you can easily access of your **own source code** base
  - A function composed **only** of calls to **functions** in **another code base**
    - BCL (File.ReadAllLines, Console, …)
    - Code elements of your programming language (for, if, ++, …)
    - Blackbox code -> Binaries (DLLs, Packages, …)
  - The leaves of the function call tree
  - **No FD**
  - Easy to test

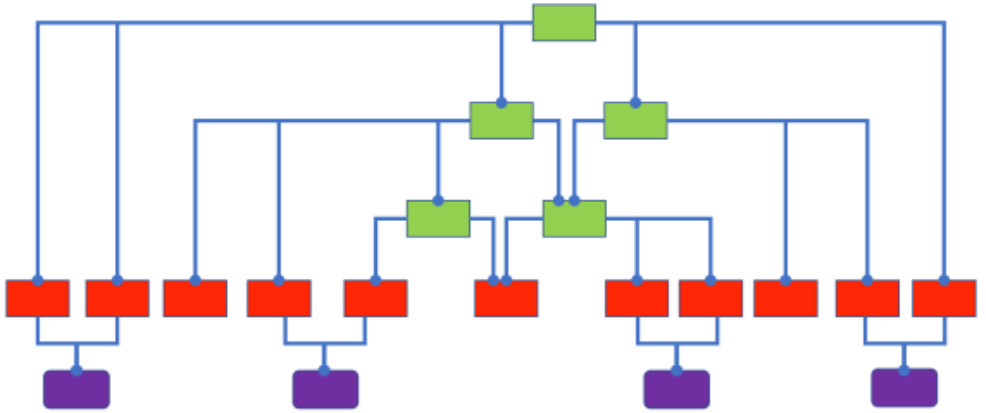


IOSP:
Integration
Operation
Segregation
Principle

- Integrations
  - Functions **without logic**
  - Integrate calls to other Integrations or Operations
  - **No FD**
  - Often the integration code is easy to understand, less tests required
  - Only a few integration tests on the top level code necessary
    - Less mocks or maybe without any mocks

New Architecture Pattern – Connect Operations

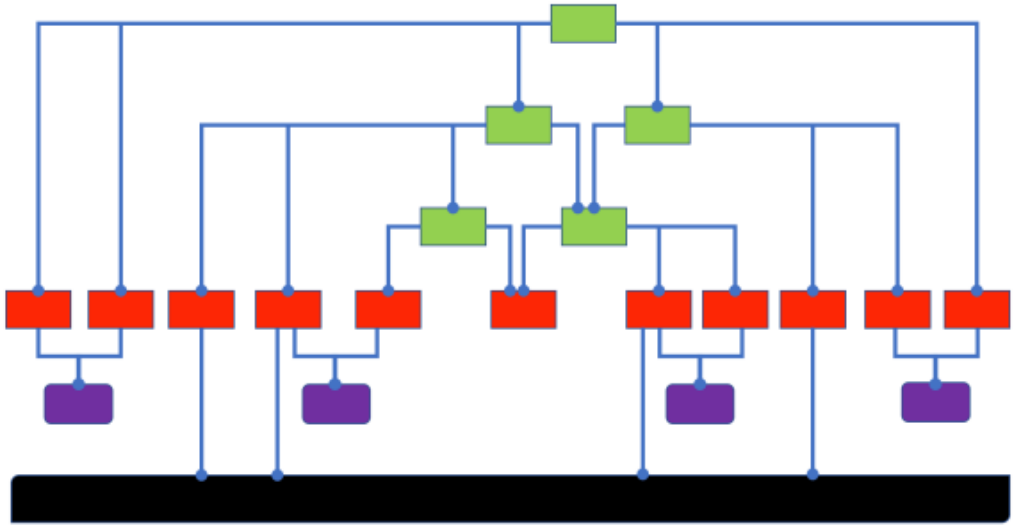
- Another problem in the common architecture models
  - No place for **DATA**
  - Not visible in the shells, layers
- Data should be explicitly addressed
- How do Operations interact/communicate when they do not know each other?
- Operations depend on **DATA**, either
  - DATA flows between them via their parent Integrations
  - or they share state (not flowing)
- Data structures / classes
  - Without methods -> No FD
  - Data classes can have Operations
    - Only Operations for data access & consistency
    - E. g. Stack, Queue, see also IODA sample (slide 33)
    - This is not a problematic FD when accessed by Operations



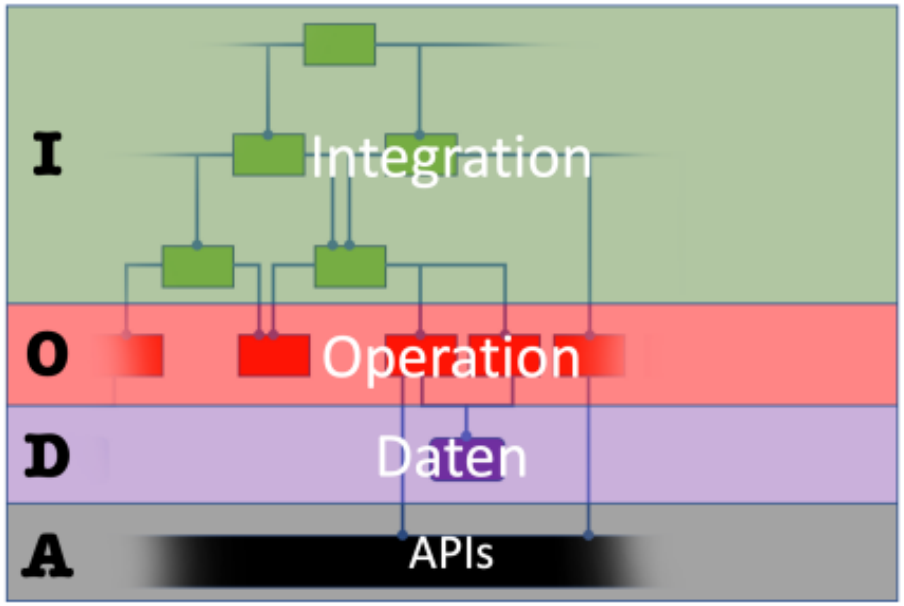
New Architecture Pattern – Connection to the Environment

- **Communication** with the **outside world** is another **special aspect** the concentric architectures have improved compared to the layer models
- I/O-Operations, API calls to libs, frameworks, external systems/services
- Problematic for tests
  - Performance
  - Effort to code
- External libs may change or may be replaced
- DIP/IoC help in such situations
- Isolate these API calls in just a few code fragments
- Concentric architectures locate this aspect at the outer shell
  - Adapter in Hexagonal Architecture
  - Controller, Gateways, Presenter in Clean Architecture

- A **new architecture** pattern should also **highlight** the **access** to the **environment** and make the **separation clear**
- Easy in a call hierarchy without FDs
- **API calls** are considered as **logic** -> **Operations**
- Dependencies to black “fundament”/**APIs** only in red Operations

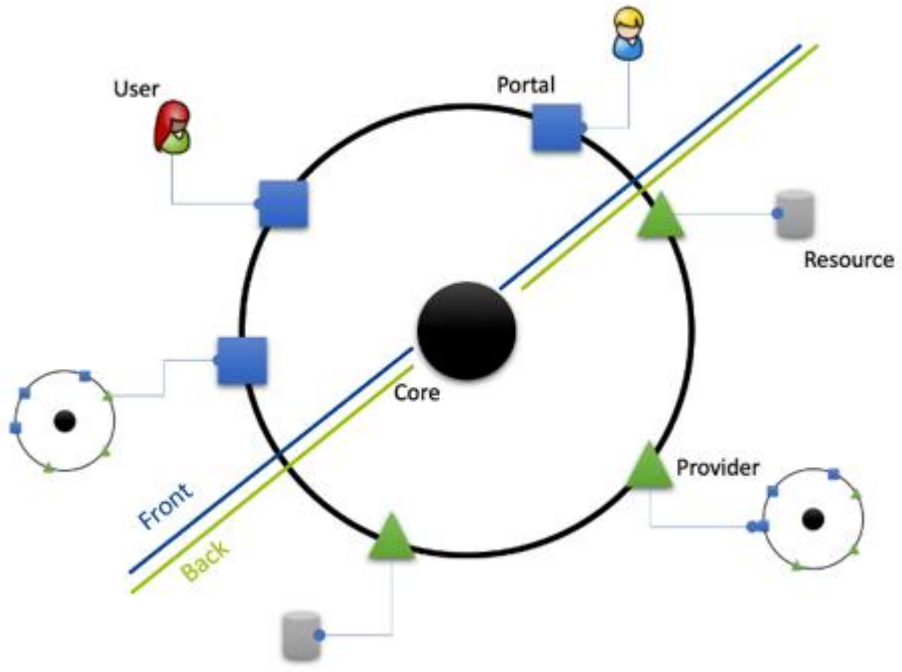


IODA – A new Architecture Pattern



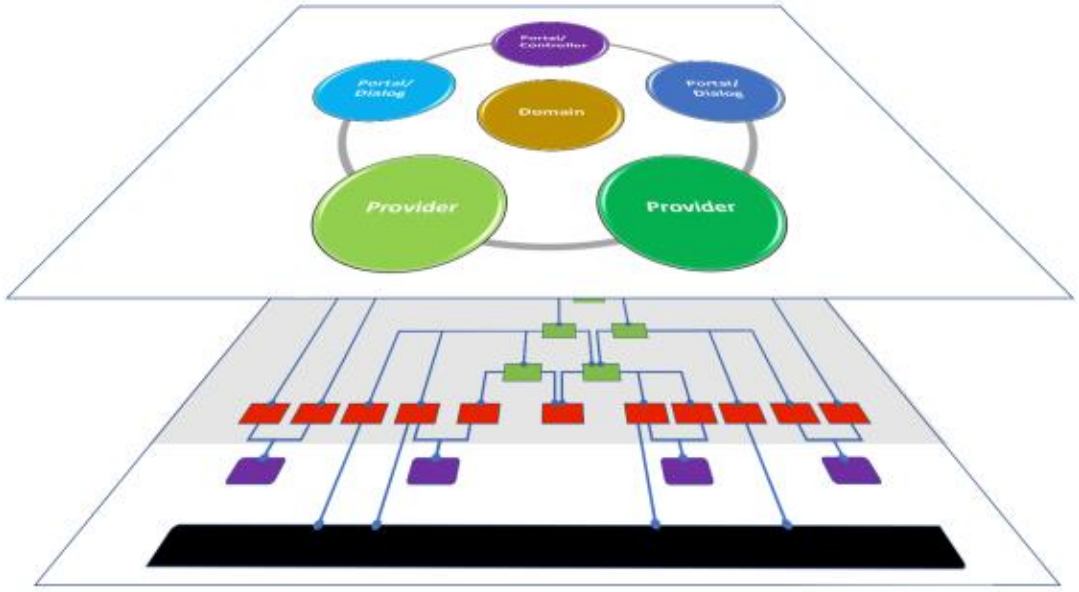
- **Generic** architecture for all kinds of software systems (Games, CRM, industrial SW, …)
  - initially no content-related responsibilities (as it is the case in the common patterns – Domain, Presenter, Gateway…), **only formal** on the 4 levels
- Defines how **code** should be **structured**
- **Focus** on **Functions**, behavior of the software, transform data
- Improves **Understandability** (Integration code easier to read)
- Improves **Testability** (Operations have less dependencies)

IODA – **Software Cell** and Content related Responsibilities



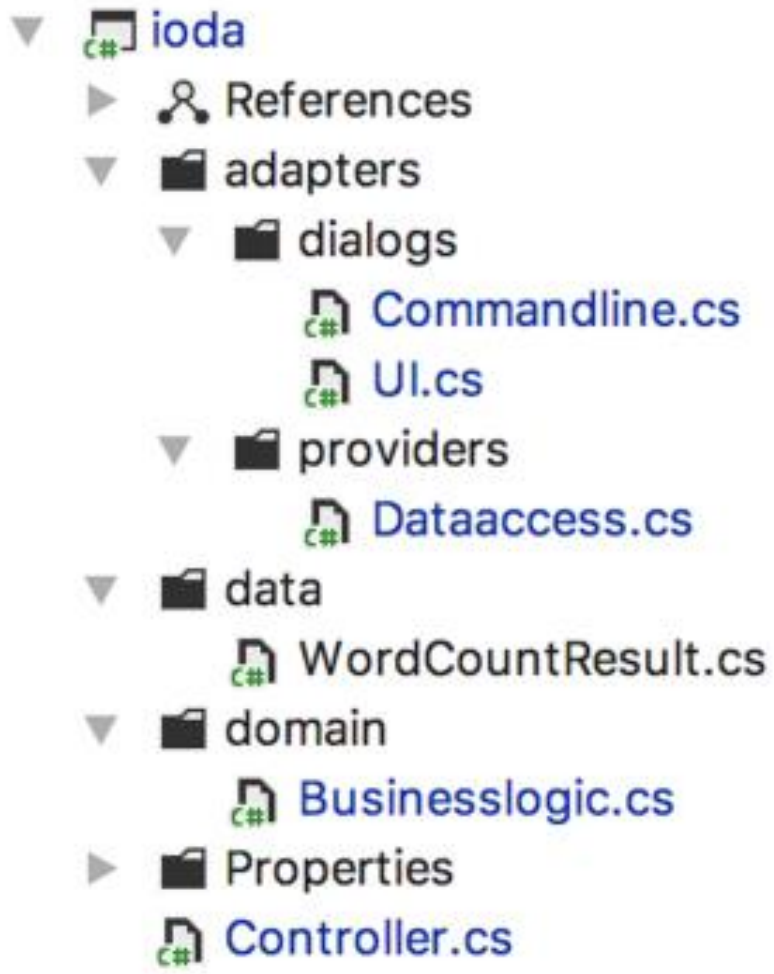
- Software **Cell**:
  - Concentric Software System
  - **Membrane**: The border to the environment (black outer circle)
  - **Core**: Domain
  - Code you write is the membrane, the core and code between both
- Fundamental responsibilities
  - At the black outer circle / **boundaries** of the system
  - Category: Adapters (blue squares, green triangles)
- **Adapter**
  - Encapsulates: IO-logic, other API calls (libs)
  - Connects the environment (users, other software cells/systems)
  - Portals and Providers
- Via **Portals** the system is **accessed by** the environment
  - Users/Other systems trigger actions and provide input data
  - Returns output data
- Via **Providers** the system **access** the resources it depends on the environment
- **No FDs!**

IODA – 2-Dimensional Architecture



- **Fundamental structure**/dimension (at the bottom)
  - Enables: **No FDs** in the above dimension
  - Hierarchy of modules/functions
  - Formal responsibilities (Integration, Operation)
  - Defines how the code is **organized**
  - Recursive: **Pattern repeats** in all parts of the software
- **Software Cell**
  - Defines system architecture
  - **Content**-related responsibilities
- Distinction between formal- and content-related responsibilities

IODA - Sample



- Adapters
  - Encapsulate APIs in portals and providers
  - Commandline: Handles “API” string[] args
  - UI: Communication with User via standard-Input/-Output
- Data
  - Connects Operations (see previous slides)
  - IODA architecture enforces to consider it
  - WordCountResult
- Entry points
  - Controller (Application)
  - Integrates Portal with the top-level/main Integrations (Processors)
  - No aspect in the common architecture models
- What´s missing?
  - Interfaces (of course needed, maybe in adapters, but less)
  - No/less DIP, less complexity, runtime vs compile time
  - IoC: Maybe to replace adapters in Integration tests

IODA - Sample

```
internal class Controller
{
    public static void Main(string[] args) {
        var cmd = new Commandline(args);
        var da = new Dataaccess();
        var ui = new UI();
        var app = new Controller(cmd, ui, da);

        app.Run(args);
    }


    private readonly Commandline _cmd;
    private readonly UI _ui;
    private readonly Dataaccess _da;

    Controller(Commandline cmd, UI ui, Dataaccess da) {
        _cmd = cmd;
        _ui = ui;
        _da = da;
    }
```

- Object initialization
  - IoC prepares for a potential DIP that may be necessary later
  - Could be done in separate Construction class, if needed with IoC-Container
- Controller / Application
  - Just integration – app.Run
- UI
  - Has no interface from the beginning
  - Could be added if needed
  - IODA model does not enforce abstraction with DIP as CA is doing it upfront. Grows with requirements

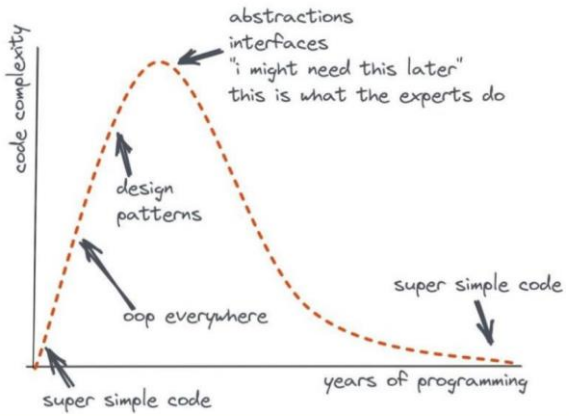
IODA - Sample

```
internal class Controller
{
    public static void Main(string[] args) {...}

    Ctor

    void Run(string[] args) {
        _cmd.Determine_text_source(
            onFile: Count_words_in_file,
            onUser: Count_words_in_user_entered_text);
    }

    void Count_words_in_file(string filename) {
        var text = _da.Load_text(filename);
        Count_words(text);
    }

    void Count_words_in_user_entered_text() {
        _ui.Ask_user_for_text(
            Count_words);
    }

    void Count_words(string text) {
        var stopwords = _da.Load_stopwords();
        var result = Businesslogic.Count_words(text, stopwords);
        _ui.Display_results(result.CountTotal, result.CountDistinct);
    }
}

class Commandline
{
    private readonly string[] _args;

    public Commandline(string[] args) { _args = args; }

    public void Determine_text_source(Action<string> onFile, Action onUser) {
        if (_args.Length > 0)
            onFile(_args[0]);
        else
            onUser();
    }
}
```

- Just Integration
- **Composition** of multiple parts
  - Creates **abstraction** on the higher functions/modules
- IODA approach is
  - **Function-first (data flow first)**
  - **Composition-first**
  - Aggregation-second: categorization via classes/modules in second step to organize functions
  - When app grows, Integrations can be moved easily to other classes
- No logic
  - **How** to **avoid logic** in Integration code?
  - **Continuations** / Delegates (see yellow parts)
  - Logic in Operations easier testable (no FDs)

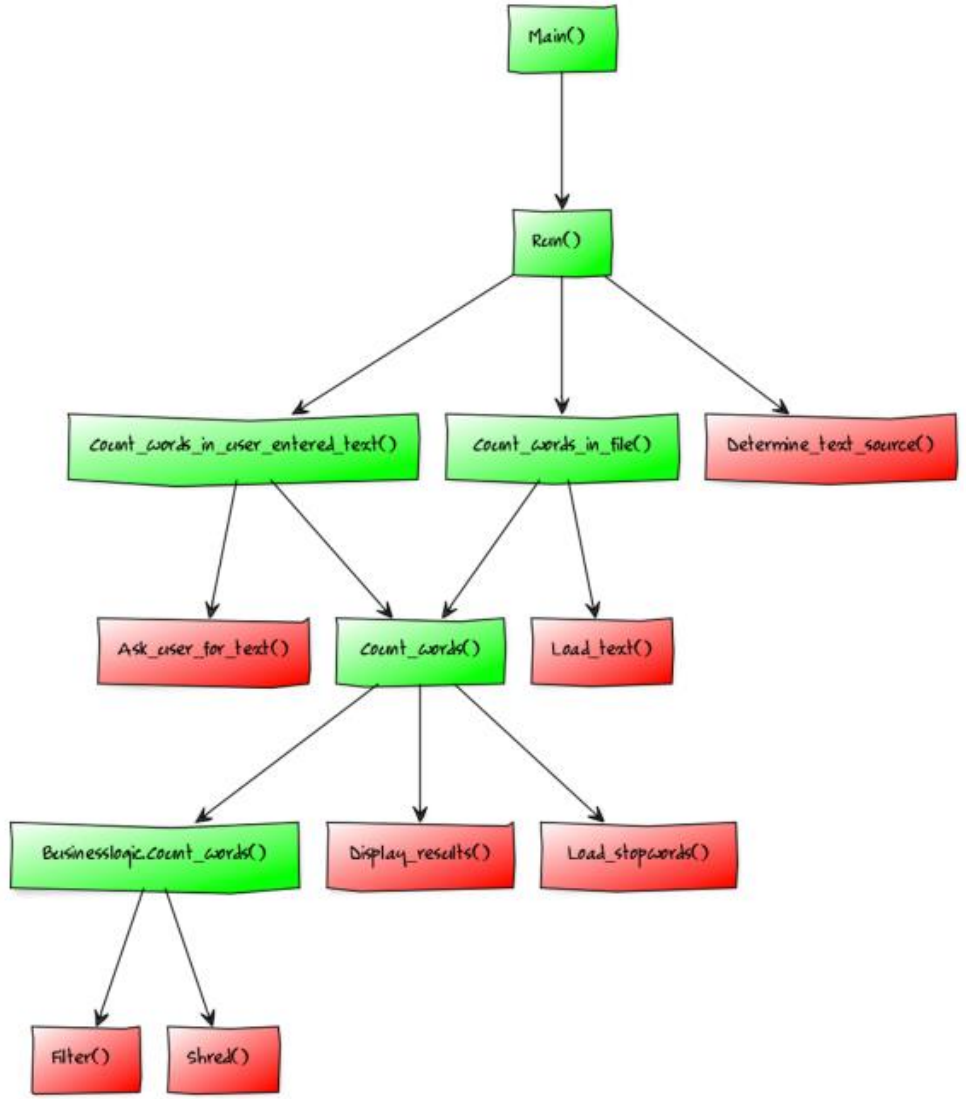
IODA - Sample

```
class Businesslogic {
    public static WordCountResult Count_words(string text, HashSet<string> stopwordsDirectory) {
        var t = Shred(text);
        return new WordCountResult {
            CountTotal = Filter(t.Words, stopwordsDirectory).ToArray().Length,
            CountDistinct = Filter(t.UniqueWords, stopwordsDirectory).ToArray().Length
        };
    }

    private static ShreddedText Shred(string text) {
        var wordCandidates = text.Split(new[] {' ', '\t', '\n', '\r'}, StringSplitOptions.RemoveEmptyEntries);
        return new ShreddedText(wordCandidates.SelectMany(Normalize));

        IEnumerable<string> Normalize(string wordCandidate) {
            foreach(var m in Regex.Matches(wordCandidate, @"[\w\-]*"))
                if (!string.IsNullOrWhiteSpace(m.ToString()))
                    yield return m.ToString();
        }
    }

    private static IEnumerable<string> Filter(IEnumerable<string> words, HashSet<string> stopwordsDirectory)
        => words.Where(w => !stopwordsDirectory.Contains(w));


    class ShreddedText {
        private readonly IEnumerable<string>  _words;

        public ShreddedText(IEnumerable<string> words) { _words = words; }

        public string[] Words => _words.ToArray();
        public string[] UniqueWords => _words.Distinct().ToArray();
    }
}
```



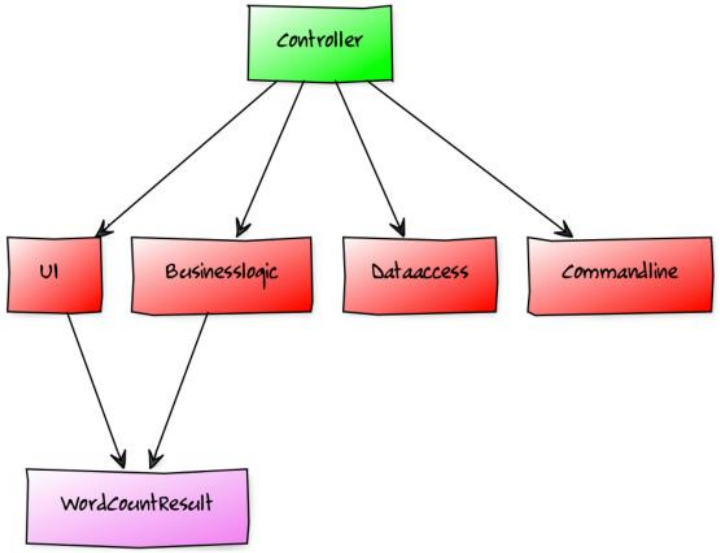
- ShreddedText
  - Data class
  - Connects Operations (Shred and Filter)
  - Also contains Operations
    - When called by other Operations -> no problematic FD
    - No need to mock such classes in tests
    - Still easy to understand, no additional complexity
    - Can be used sometimes to move logic to data class to avoid mix of integration and logic code
- What else to see?
  - BL is **static**. Why?
  - **No signal** for instance methods
    - No common state
    - No API calls e. g. to data access like in other models, nothing to mock in this case
  - **Easy** to **test** the logic
  - Improved **understandability** (no instantiation, no shared state)
  - **No interface** necessary (maybe in future)

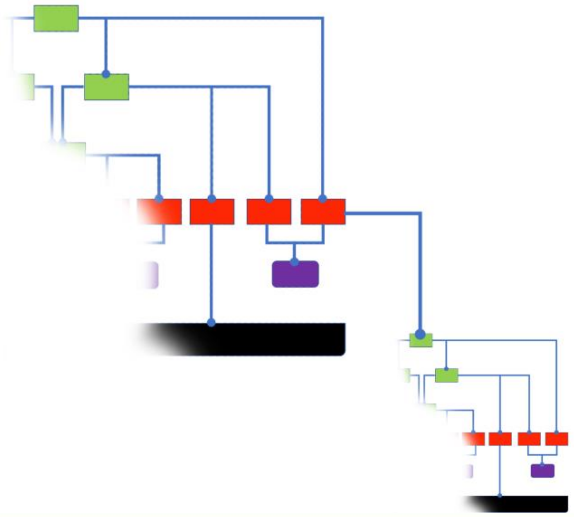
IODA Structure - Functions



- Green functions
  - without logic
  - Referencing other functions
- Red functions
  - Logic
  - Leaves
  - No outgoing arrows / no references to the sources of your solution
- Typically
  - Integrations compose 3-5 functions
  - When growing -> compose new Integration and push the others to it on a lower level
  - Sometimes the opposite needed – lift lower methods up and remove their previous Integration
  - When logic in Operation needs to be extracted the original Operation **becomes** an Integration -> “**Refactoring to IOSP**”

IODA Structure - Classes





- Green classes
  - Integration character / less or no logic
  - Referencing other classes
- Red classes
  - Logic
  - BL discrepancy? Mix of Integration and Operation
  - More Operational character (see Sample Count_words on previous slide)
- Purple class
  - Data
- IODA architecture model is **recursive**
  - Operation can call other Integration when in binary format (package reference see slide FD)
  - Or Integration calls other Integration which integrates Operations

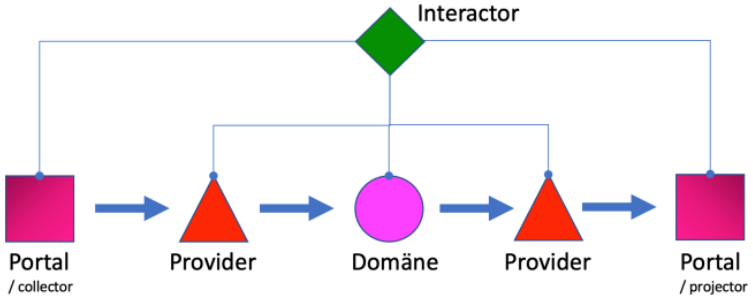
Special Integration Patterns

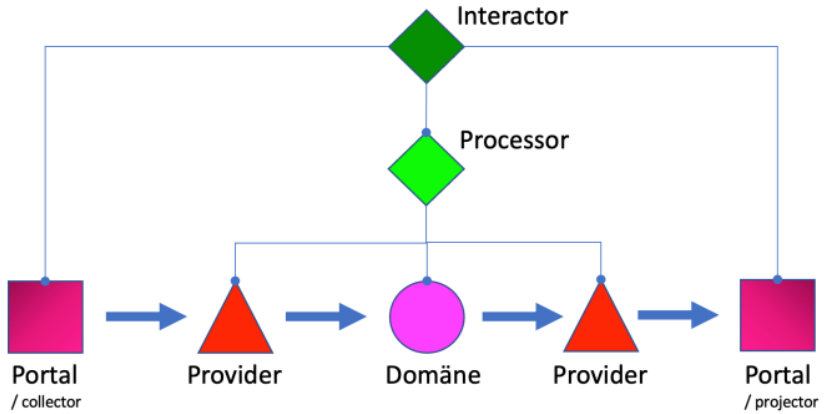
Rough process of creating behavior and data flow

- Portal receives data from environment
- Additional data may be collected from Provider
- Domain is processing the data and storing it via Provider
- Environment gets data presented
- NOTE: **Functions** do **not know** each **other**

Integration needed

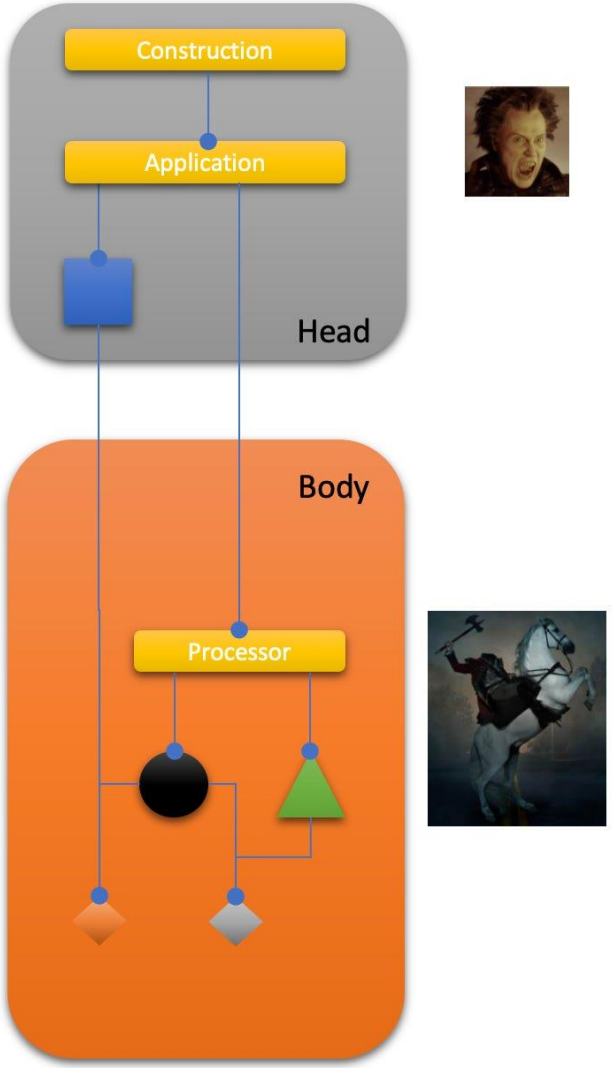
- Provider and Domain Core are the “work horses”
- Portal collects / projects data
- Integration needed -> Interactor
- **But** Interactor not testable (dependency to Portal: UI, web framework)





- Interactor variations
  - Application (Desktop app)
  - Controller (Web)
- Processor
  - The heart of IODA
  - Acceptance tests can start here
  - No UI/web framework dependencies
  - Can be reused in any other application

Finally - Sleepy Hollow Architecture



- Head
  - Integrates Portal and Body via Application
- Body
  - Integrates the Domain core and Providers via Processors
- Head and Body can be hosted in different threads, processes

Summary

- Keeps the advantages and removes the disadvantages from the concentric architecture models
- **Software Cell**
  - Detect and **locate** default **responsibilities (Adapters)**
  - Identify environment communication
  - Protect the domain core from dependencies on peripheral aspects, e. g. 3rd party libs, IO-technologies (UI, DB, network, file access,…)
- **Design**-time and **runtime** relationships are different
- **IOSP**: **Avoid FDs** to improve **changeability** and **testability**
- **Reduce** complexity of **DIP** / IoC, less interfaces, only used where necessary
- **Data** gets visible in the **model**
- **Categorization** of software modules / classes arise naturally
  - not upfront
  - functions and compositions first (compose/integrate functions to create new behavior)
  - aggregation second (organize similar things together)
- Can be used as a basic model in **any software**
- Even in brown field it can help
  - Use IODA structure as a target or **NorthStar** for refactoring and new features

Links / Resources

- Slides based on blogs, publications and articles by Ralf Westphal (Coach, Consultant, Speaker in Germany/Bulgaria)
  - “Die IODA Architektur im Vergleich”-Update 2020 (dnp dotnetpro magazine 2018-2020)
  - https://www.dotnetpro.de/hefte_data_1550430.html
  - https://www.dotnetpro.de/hefte_data_1557946.html
  - https://www.dotnetpro.de/hefte_data_1579489.html
  - Book http://leanpub.com/ioda-architektur-im-vergleich-dnp
  - Related books
    - Test-first Codierung - Programming with Ease - Teil 1
    - Softwareentwurf mit Flow-Design - Programming with Ease - Teil 2
  - English content
    - https://ralfwestphal.substack.com/p/ioda-architecture
    - Sleepy Hollow Architecture - No application should be without it - ralfw-de
    - https://ralfwestphal.substack.com/s/clean-code-development
    - https://ralfwestphal.substack.com/p/iosp-20
    - https://ralfwestphal.substack.com/p/functional-dependencies
    - https://github.com/ralfw/architecture-variations
    - Terminus Architecture - ralfw-de
- Other resources with similar approach
  - A-Frame Architecture